
Sky Transport

Shipment Document Data Extractor

A web application that reads a Bill of Lading (BOL) or Proof of Delivery (POD) — a PDF or a phone photo — and extracts the shipment fields a dispatcher would otherwise retype by hand.

Project track	AUTOMATE — Sky Transport Solutions technical assessment
Author	Siddharth Bhamare
Report date	August 2026
Live application	https://sky-transport-project.vercel.app
Backend API	https://sky-transport-project.onrender.com
Source repository	github.com/SiddharthBhamare01/Sky_transport_project
License	MIT

Executive Summary

This project automates a small but recurring manual task at a trucking/logistics operator: retyping shipper, consignee, load number, dates, weight, and commodity fields off of paper or photographed Bill of Lading (BOL) and Proof of Delivery (POD) documents into a tracking spreadsheet. Rather than building a brittle OCR/template pipeline, the solution uses a multimodal LLM call constrained to a strict JSON schema, keeping a human in the loop for every field before it is saved. The application is fully deployed and live: a FastAPI backend on Render, a static frontend on Vercel, and Supabase providing authentication and a shared, multi-user Postgres-backed queue.

The result is a working, deployed, mobile-capable tool — not a prototype — including real accounts, password reset, email notifications, camera-capture support for field use, and a CI pipeline, alongside an honest account of where the extraction's confidence-flagging still falls short (see Testing & Results).

Problem Statement

At a small trucking/logistics operator, dispatchers regularly retype the same handful of fields off of BOL and POD paperwork — PDFs from carriers, or phone photos taken at the dock — into a tracking spreadsheet. It is the same few minutes of manual copying per document, several times a day, and it is exactly the kind of repetitive task that is easy to get wrong under time pressure: a transposed digit on a load number is a real problem when someone goes looking for that shipment later.

Solution Overview

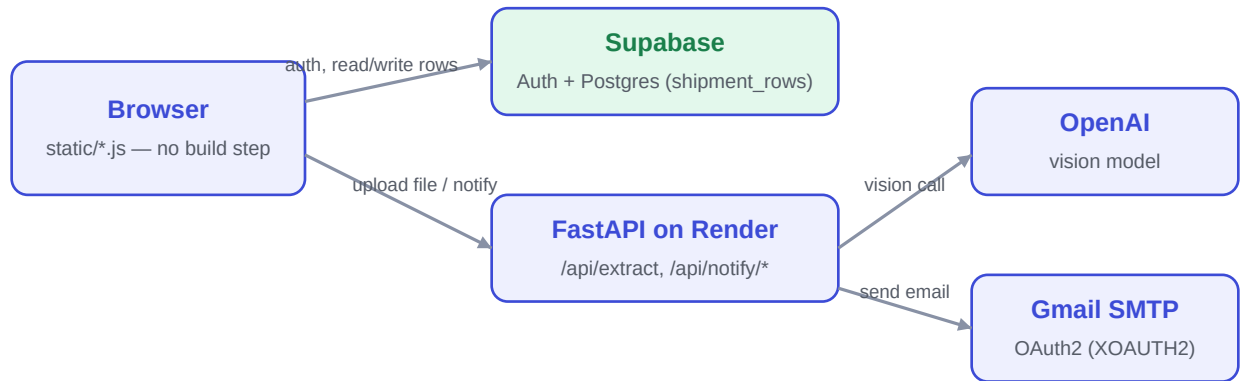
Rather than a rigid template-matching OCR pipeline — brittle across different carriers' document layouts, and slow to harden in the time available — the solution uses a multimodal LLM call constrained to a fixed JSON schema. This gets layout-agnostic extraction "for free" via prompting, while the strict schema keeps the output predictable enough to build a UI around. The design leans on keeping a human in the loop: the model is instructed to say "I don't know" rather than guess, and every field stays editable before anything is saved.

Key features

- LLM-based extraction with a strict JSON schema — no brittle OCR/regex templates.
- Three ways to get a document in: drag-and-drop, file browser, or a dedicated camera-capture button for mobile field use.
- HEIC/HEIF support — the default photo format on iPhones, transcoded server-side before extraction.
- Per-field low-confidence flagging, not just one document-level flag.
- Accounts and a shared queue — Google or email/password sign-in via Supabase Auth, with password reset.
- Transactional email notifications (welcome, password-changed, shipment-added) sent via Gmail over OAuth2, always to the token's own verified address — never a free-text recipient.
- Sample/demo mode — three fixture documents let anyone demo the full workflow with zero API calls.
- CSV export, client-side, ready to paste into a spreadsheet.
- Mobile-first refinements: ~44px touch targets, offline/slow-network-aware error messages, and loading feedback for the backend's free-tier cold start.

Architecture & Tech Stack

The frontend talks to Supabase directly for data (authentication, reading/writing shipment rows) using a public, Row-Level-Security-protected key. The FastAPI backend stays stateless and is responsible for exactly two things: calling OpenAI, and sending the three notification emails — which need secrets that must never reach the browser.



Request flow between the browser, Supabase, the FastAPI backend, and external services.

Layer	Choice
Backend	Python, FastAPI, Uvicorn
Extraction	OpenAI vision model (gpt-4o-mini by default), structured JSON-schema output
Document handling	PyMuPDF (PDF → image), Pillow + pillow-heif (HEIC → PNG)
Frontend	Vanilla HTML/CSS/JS — no build step, no framework
Auth & database	Supabase (Postgres + Auth: Google OAuth and email/password)
Email	Gmail SMTP via OAuth2 (XOAUTH2), verified with a Supabase JWT server-side
Hosting	Render (backend API), Vercel (static frontend)
CI	GitHub Actions — backend import/compile check, frontend JS syntax check

Application Walkthrough

Screenshots below are from the live, deployed application (stubbed session for illustration).

Sign in

Email/password or Google, via Supabase Auth. Login is required to use the extractor, which is what allows every row in the shared queue to carry a genuine, verified owner.

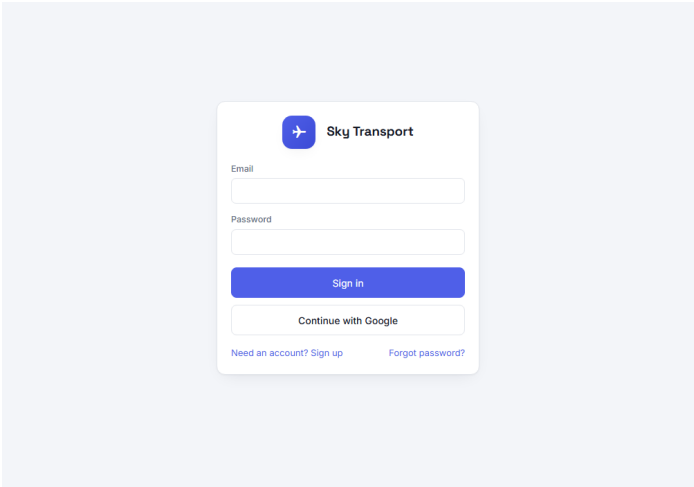


Figure 1 — Sign-in screen

Extraction results and per-field confidence flagging

The screenshot below demonstrates the core feature in action: **delivery_date** and **signature_present** are individually highlighted because the source document doesn't clearly show them, with a plain-language explanation banner above the form. The shared queue table below carries the same flag through on a saved row, so a dispatcher scanning the table later can still see which rows need a second look.

**Shipment Document Data Extractor**

dispatcher@example.com [Log out](#)

Upload a Bill of Lading or Proof of Delivery — extract the fields, review, and export to CSV.



Drop a file or [browse](#)

PDF, PNG, JPG, HEIC · up to 10MB

Extract

OR

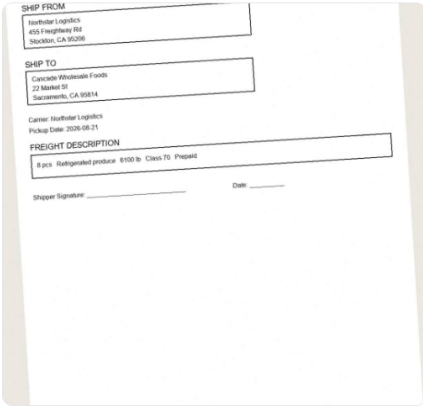
No document handy? Try a sample — same full workflow, no upload needed:

Clean BOL (PDF) — Acme Freight

Phone-photo BOL (JPG) — Northstar Logistics

Proof of Delivery (PNG) — Acme Freight

Document preview



Extracted fields

⚠ Review recommended: Field 'delivery_date' is missing. Handwriting is clear but the date field is blank. Signature field is empty, indicating no signature is present.

Doc type

BOL

Shipper

Northstar Logistics

Shipper addr

455 Freighthway RdStockton, CA 95206

Consignee

Cascade Wholesale Foods

Consignee addr

22 Market StSacramento, CA 95814

Carrier

Northstar Logistics

Load #

LD-77120

PRO #

PRO-114455

Pickup date

2026-08-21

Delivery date ⚠

Weight

6100

Weight unit

lb

Pieces

8

Commodity

Refrigerated produce

Terms

prepaid

Signed? ⚠

false

Add to table

Accumulated shipments (3)

Clear all

Export CSV

DOCUMENT_TYPE	SHIPPER_NAME	SHIPPER_ADDRESS	CONSIGNEE_NAME	CONSIGNEE_ADDRESS	CARRIER
BOL	Acme Freight LLC	1200 Industrial Pkwy, Modesto, CA 95351	Bluepeak Retail Co.	88 Commerce Dr, Reno, NV 89501	Acme F
BOL	Northstar Logistics	455 Freightway Rd, Stockton, CA 95206	Cascade Wholesale Foods	22 Market St, Sacramento, CA 95814	Northst
POD	Acme Freight LLC		Harbor & Vine Cafe Supply		

Figure 2 — Extraction results with low-confidence flags and the shared queue

Mobile field use

On a phone, a dedicated camera-capture button (using the HTML capture attribute) opens the rear camera directly, and HEIC photos — the default format on iPhone — are supported end to end. Interactive elements are sized to the ~44px minimum recommended touch target.

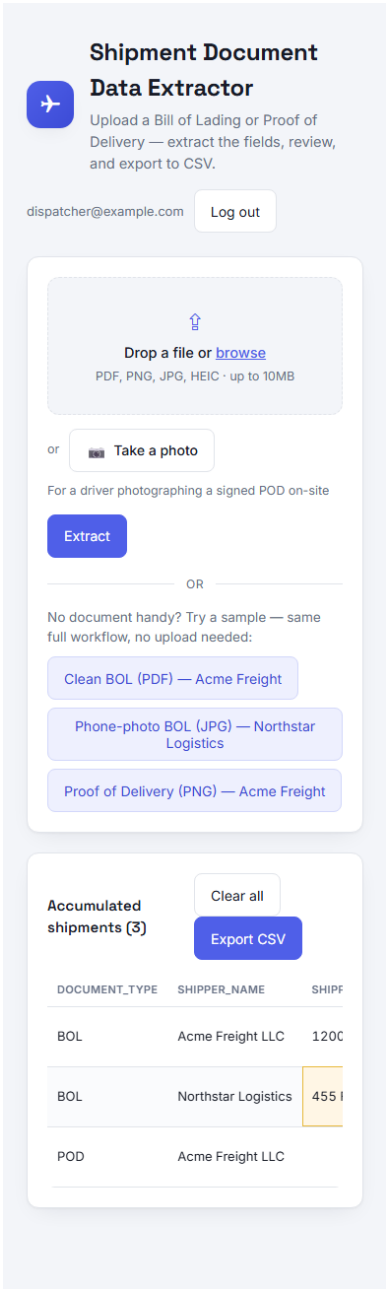


Figure 3 — Mobile view with camera capture

Testing & Results

All three sample documents were run through the real pipeline (gpt-4o-mini, live API key, not mocked) twice — once with the original single-flag design, once after adding `low_confidence_fields`.

Run 1 — single review_recommended flag

The clean printed PDF extracted perfectly. The POD also extracted cleanly, correctly leaving fields not present on a POD as null. The noisy/skewed "phone photo" fixture is where it broke: the model misread "Freightway Rd" as "Freighthay Rd" and did **not** set `review_recommended` to flag it — a dispatcher trusting that flag would have missed a bad row.

Run 2 — after adding low_confidence_fields

The clean PDF still extracted perfectly with nothing flagged. The fix did not fully work, though: on a re-run of the noisy photo, the model produced yet another misread of the same street name ("Freighthway Rd", a third variant across runs, confirming the model is non-deterministic here) and again did **not** flag `shipper_address` as low-confidence. Separately, on the POD it over-flagged: six fields correctly left null (because they are not on a POD at all) were also listed as low-confidence, conflating "structurally absent" with "illegible."

This is an honest, partial result rather than a fixed one — the feature changed the failure mode without eliminating it. A second verification pass on flagged fields, or separating "not applicable" from "illegible" directly in the schema, is the recommended next iteration rather than trusting the current flag as-is.

Additional verification performed this session

- A HEIC test photo was generated and run through the real extraction pipeline end to end (direct call and full HTTP path), correctly matching ground-truth field values.
- Camera-capture wiring was verified with a stubbed-session Playwright test plus a real local extraction round trip, since no physical phone camera was available to test with directly.
- Mobile layout, touch-target sizing, offline handling, and the Render cold-start loading indicator were each verified with targeted automated browser tests, not just visual inspection.
- Three additional fictional test PDFs and a QA guide document (test-samples/) were produced for exercising the live upload path independently of the built-in demo fixtures.

Known Limitations & Future Work

- Single/small documents only — no multi-page batch handling.
- Per-field confidence flagging is a partial fix, not a full one (see Testing & Results above) — a second verification pass or schema-level separation of "not applicable" vs. "illegible" is the recommended next step.
- PDF preview uses an iframe; browsers without a built-in PDF viewer fall back to an "Open PDF in new tab" link.
- No mobile "card view" for the shared table yet — on a narrow phone screen it is a horizontally-scrollable wide table rather than a stacked layout.

- No offline draft persistence — a backgrounded mobile tab discarded by the OS mid-review will lose in-progress, not-yet-saved field edits.
- Session-expiry mid-review surfaces as a generic save error rather than an automatic re-login prompt that preserves the in-progress review.
- With more time: batch upload, duplicate/date-sanity validation against existing rows, and a direct export/integration into whatever system the dispatch team already uses instead of a CSV hop.

Deployment

The application is deployed with continuous delivery already wired up: Render and Vercel both auto-deploy from the main branch on every push, and a GitHub Actions workflow runs a build/syntax gate on every push and pull request, independent of the deploy itself.

Component	Platform	Notes
Frontend	Vercel	Static site, Root Directory set to static/, no build step
Backend API	Render	Free tier — cold-starts (~50s) after inactivity
Database & Auth	Supabase	Postgres + Auth (Google, email/password), Row Level Security enforced
Email delivery	Gmail (OAuth2)	Backend-only secrets, never exposed to the browser
CI	GitHub Actions	Runs on every push/PR, independent of the native Render/Vercel deploy

Conclusion & Key Takeaways

- A schema-constrained multimodal LLM call is a practical substitute for a template-matching OCR pipeline on documents whose layout varies by sender — it generalized across three visually distinct fixtures (a clean PDF, a skewed/noisy phone photo, and a structurally different POD) without any per-layout code.
- Self-reported confidence is not reliable on its own: a single document-level "review recommended" flag missed a genuine misread in testing. Reframing it as a legibility check on individual fields (`low_confidence_fields`) changed, but did not eliminate, that failure mode — evidence that a second verification pass or a stricter schema distinction is needed before this flag should be fully trusted.
- Keeping a human in the loop — every field editable before saving, null over guessing — is a deliberate design choice that limits the damage of the extraction model's remaining unreliability, rather than a workaround for it.
- Adding accounts was driven by a concrete constraint, not scope creep for its own sake: a safe "who gets notified" answer required a real, verified identity, which a stateless, database-free design could not provide.

Illustrative business-impact estimate

Using representative assumptions rather than measured data: if manually transcribing one document takes a dispatcher roughly 3 minutes, and a small dispatch team processes on the order of 20 such documents per day, automating extraction and review down to under a minute per document would save on the order of **~40–50 minutes per day**, or roughly **3–4 hours per week**, for that team — before accounting for the reduction in transcription errors that a human-reviewed, schema-validated pipeline provides over manual retyping.

Full technical documentation, setup instructions, and the complete honest testing narrative are maintained in the project's README.md in the source repository linked on the cover page.